

Website Analytics Engineering Guide

Public repository technical guide for maintainers and contributors

Last changed: July 23, 2026

Public repository edition draft

Table of Contents

- Table of Contents 2
- Engineering goals 3
- Source module map 3
- Runtime data model..... 3
- SQLite schema and public repository policy 3
- Authentication design 4
- GA4 request design 4
- UI engineering notes 4
- Build and smoke test..... 5
- Open-source hardening checklist..... 5
- Contribution workflow 5
- Future extension points 6

Engineering goals

- Keep the dashboard simple, fast, and readable.
- Keep FMX forms editable in the Delphi IDE. Avoid runtime-only UI construction unless the maintainer explicitly approves it.
- Keep analytics results memory-only. Persist only local settings and user-owned OAuth refresh material.
- Keep the public repository safe: no local databases, no secrets, no generated build output, no personal paths.

Source module map

Module	Responsibility
WebsiteAnalytics.MainForm.pas/.fmx	Main dashboard, filters, summary cards, grids/charts, update flow, status, help/settings buttons.
WebsiteAnalytics.GA4DataModule.pas	GA4 Data API requests, response parsing, standard report and realtime retrieval.
WebsiteAnalytics.AuthenticationDataModule.pas	OAuth desktop flow, loopback listener, token exchange, refresh-token retry, DPAPI protection.
WebsiteAnalytics.SettingsDataModule.pas	SQLite schema creation, defaults, and read/write access.
WebsiteAnalytics.SettingsForm.pas/.fmx	OAuth and dashboard settings UI, masked secret handling, connect/disconnect commands.
WebsiteAnalytics.PropertyManagerForm.pas/.fmx	Website property CRUD and property ID management.
WebsiteAnalytics.DiagnosticsForm.pas/.fmx	Diagnostics and self-test surface.
WebsiteAnalytics.Models.pas	Shared record/class models for GA4 rows and snapshots.

Runtime data model

The app separates persistent settings from transient analytics data. GA4 results are assembled into an in-memory snapshot and used to populate the dashboard. Closing the application discards those report rows.

- Persistent: website_properties, app_settings, OAuth setup values, saved grid widths, encrypted refresh token.
- Transient: KPI summaries, trend points, geography rows, content rows, realtime rows, chart data, status messages.

SQLite schema and public repository policy

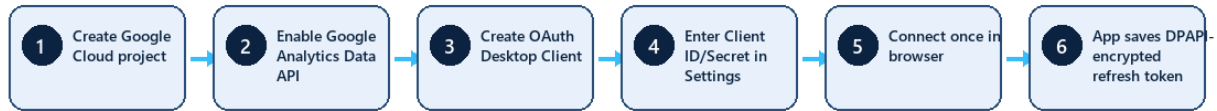
The schema file is tracked so a clean checkout can create its local database. The generated database file is ignored.

Table	Important columns
app_settings	setting_name, setting_value, setting_group, description, updated_utc
website_properties	display_name, website_address, ga4_property_id, display_color, enabled, display_order

- databases/WebsiteAnalytics.schema.sql may be tracked.
- databases/*.sqlite3 must be ignored.
- Before public release, replace any maintainer-specific default property IDs with placeholders, sample IDs, or documented demo values as appropriate.

Authentication design

Google OAuth and GA4 setup flow



Public repository rule: every programmer creates and enters their own Google OAuth desktop credentials.

Do not commit credentials, databases, tokens, or credential JSON files.

OAuth architecture for public users.

Authentication uses Google OAuth for installed/desktop applications. The app starts a local loopback listener, opens the user browser, receives the authorization code, exchanges it for tokens, and stores the refresh token encrypted with Windows DPAPI. The access token remains in memory.

- Scope: <https://www.googleapis.com/auth/analytics.readonly>.
- Client ID and secret are user-provided settings, not repository secrets.
- Refresh token setting: `ga4_oauth_refresh_token_dpapi`.
- Silent startup reconnect decrypts the saved refresh token for the current Windows account and refreshes the access token.
- If refresh fails, the app falls back to interactive Google connection from Settings.

GA4 request design

The GA4 data layer requests standard report rows for the selected property/date range and realtime rows for current activity. Reports should remain narrowly focused on the dashboard questions rather than attempting to mirror every GA4 screen.

Dashboard area	Typical GA4 dimensions/metrics
Locations	country, region, city; activeUsers, sessions
Pages/downloads	pageTitle, pagePath, eventName; activeUsers, eventCount
Users over range	date; activeUsers
Realtime	city, country or region; activeUsers and recent activity fields where available
Summary cards	aggregated users, views, events/downloads, top page, realtime counts

UI engineering notes

- The main form is FMX and should remain editable in the Object Inspector.
- When layout behavior truly requires code, keep it narrow, explain why, and avoid unnecessary helper code.
- Charts are custom painted and support hover tooltips for exact values.
- Grids persist user-adjusted column widths through settings.

- The UI should remain reasonably resizable for common desktop resolutions.

Build and smoke test

1. Open a RAD Studio command prompt or call rsvars.bat from C:\Program Files (x86)\Embarcadero\Studio\37.0\bin, adjusted for the public user's installed version.
2. Build Win64 Debug with msbuild WebsiteAnalytics.dproj /t:Build /p:Config=Debug /p:Platform=Win64.
3. Build Win32 Debug if that target is supported.
4. Smoke-test startup with -no-startup-auth so the form loads without forcing Google sign-in.
5. Verify no FMX streaming errors such as missing properties or invalid property values.

Open-source hardening checklist

Public repository safety model

Track in Git

- ☐ Delphi source and editable FMX forms
- ☐ README, guides, help files
- ☐ databases/WebsiteAnalytics.schema.sql
- ☐ images/icons intended for distribution
- ☐ .gitignore and clean build scripts

Never track in Git

- ☐ databases/*.sqlite3
- ☐ OAuth client secrets or refresh tokens
- ☐ credential JSON, .env, PEM, key files
- ☐ personal machine paths or user profiles
- ☐ bin/dcu/generated build output

Repository safety checklist for maintainers.

- Review .gitignore before public release.
- Search for OAuth secrets, refresh tokens, sqlite files, credential JSON, personal folders, and private URLs.
- Use a clean public repository if private history ever contained secrets.
- Do not commit screenshots showing unredacted secrets.
- Use placeholders in docs: YOUR_OAUTH_CLIENT_ID, YOUR_OAUTH_CLIENT_SECRET, YOUR_GA4_PROPERTY_ID, and <https://github.com/<your-user>/<repo>>.

Contribution workflow

6. Create a branch for changes.
7. Make a pre-change backup when working under the maintainer's local directives.
8. Keep edits scoped.
9. Build and smoke-test.
10. Stage only intended files.
11. Commit with a clear message.
12. Push and open a pull request when the repository is public.

Future extension points

- Traffic sources/referrers.
- Devices, browsers, and operating systems.
- Language/localization reports for worldwide users.
- Data quality reporting for not-set locations and missing dimensions.
- Optional exports, if the app later needs CSV/PDF reporting.